

MODUL 8

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

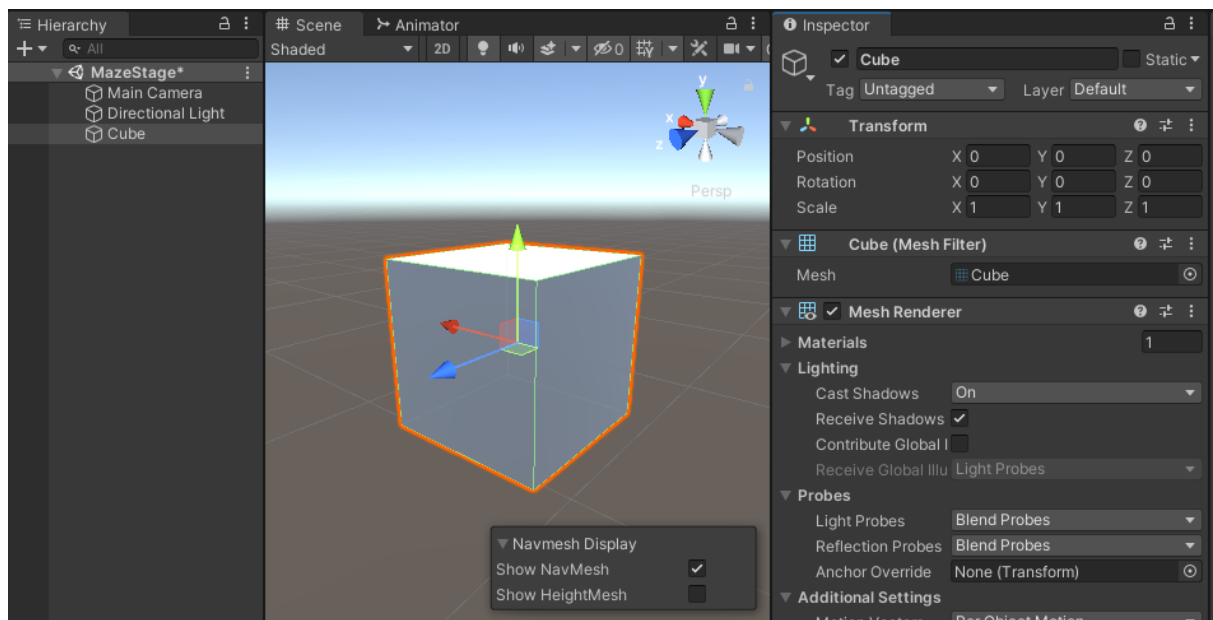
Procedural Maze Generation (Drunken Crawl Algoritma)

Pada Modul ini, Mahasiswa akan mempelajari tentang Algoritma dasar Maze Generation, algoritma ini sangat membantu proses development game dengan multiple level, karena semua level design dilakukan oleh perhitungan matematis atau AI, oleh karena itu algoritma Procedural Maze & Dungeon Generation ini akan mempercepat proses development sebuah game. Berikut adalah penjelasan lebih detail tentang maze generation.

1. Algoritma Maze Generation.

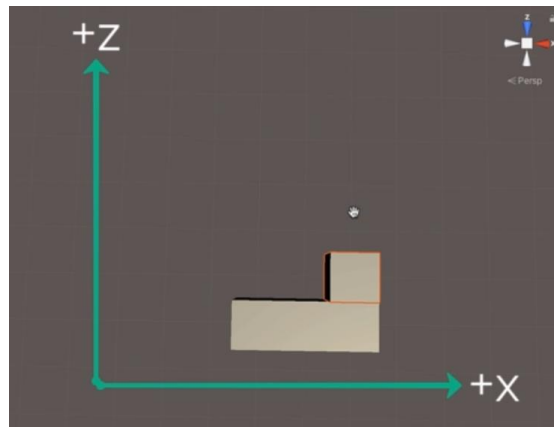
1.1. Object Dasar Maze Generation

Implementasi Maze generation pada unity akan menggunakan object **Cube** berskala **Vector3(1,1,1)** dan berada pada posisi **Vector3(0,0,0)**, hal ini bertujuan untuk memudahkan proses perhitungan matematis.

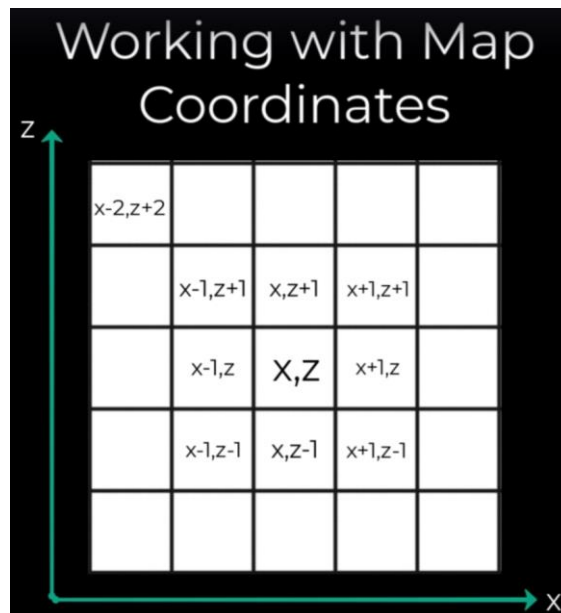


1.2. Sumbu Orientasi Maze Generation dan Transformasi Koordinat

Pembuatan Maze generation memerlukan 2 sumbu 2D, pada unity akan digunakan sumbu Z dan sumbu X, Sumbu Y tidak digunakan karena maze tidak memerlukan variable Tinggi.

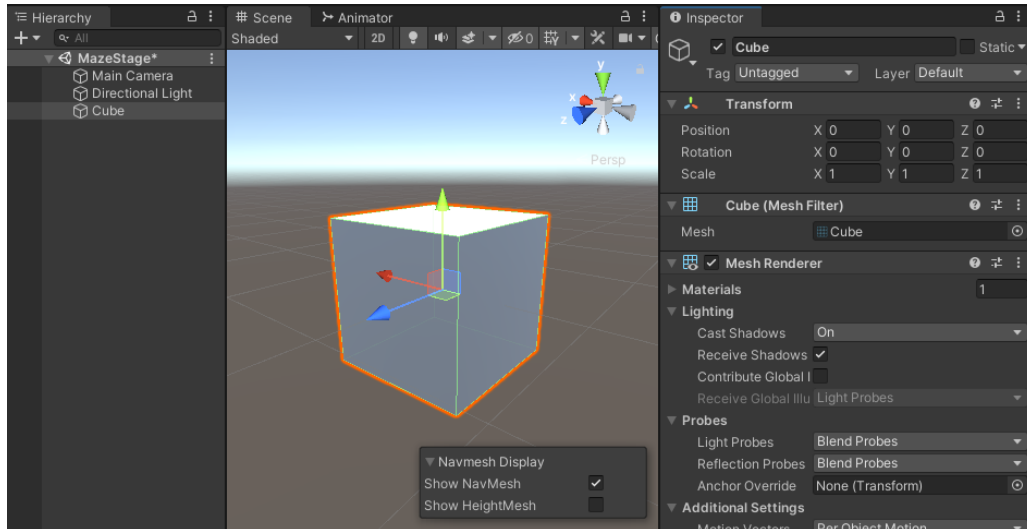


Transformasi Object yang akan dilakukan adalah menambah/mengurangi value sebesar 1. Misalnya titik awal adalah **vector3(0,0,0) atau X,Z**, dan akan digeser kekanan 1x, maka akan dirubah menjadi **vector3(1,0,0) atau X+1,Z**. Pada gambar dibawah ini adalah

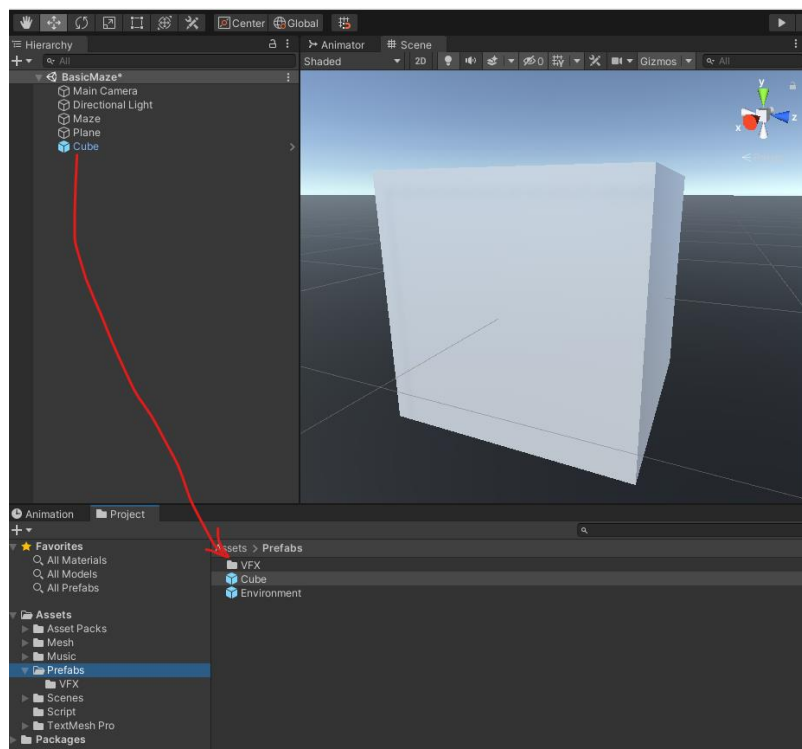


2. Implementasi Modular Maze

- Buat **Scene Baru** (Nama Bebas) dan Create **Object Cube** dengan dimensi skala $\text{Vector3}(1,1,1)$ seperti pada gambar dibawah ini



- Buat **Prefab baru** dari **Object Cube** yang sudah dibuat sebelumnya. Dengan cara **drag and drop** GameObject Cube kedalam Project dengan directory **Assets/Prefabs**, setelah itu **Delete Cube** yang ada di dalam **Hierarchy**.



- Buat Empty GameObject Bernama **MazeManager** dan tambahkan Script Baru Bernama MazeLogic, dan tulis beberapa baris kode berikut.

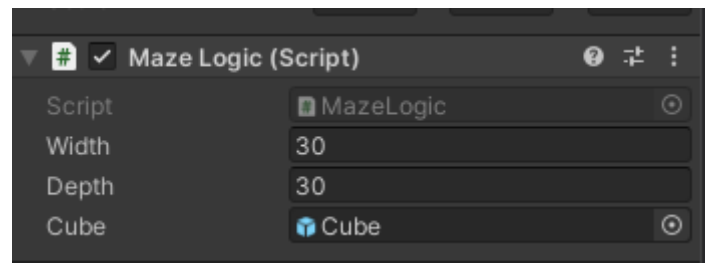
```

Unity Script (1 asset reference) | 0 references
public class MazeLogic : MonoBehaviour
{
    public int width = 30; //x length
    public int depth = 30; //z length
    public GameObject Cube; //Maze Wall

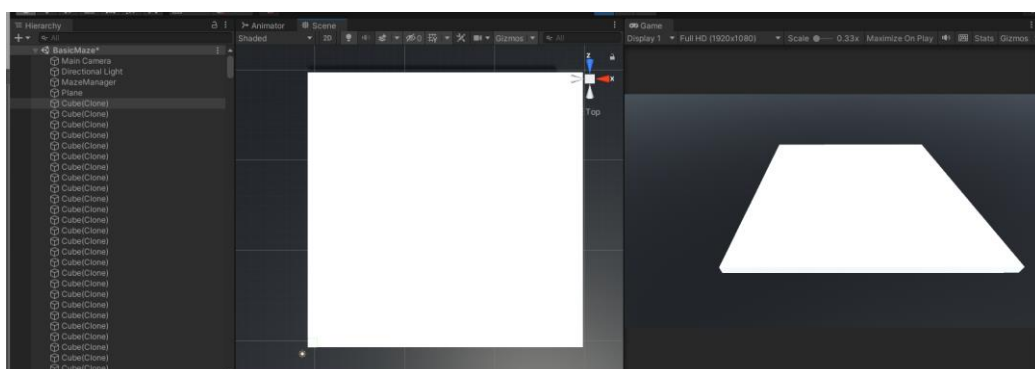
    Unity Message | 0 references
    void Start()
    {
        for (int z = 0; z < depth; z++)
            for (int x = 0; x < width; x++)
            {
                Vector3 pos = new Vector3(x, 0, z);
                GameObject wall = Instantiate(Cube, pos, Quaternion.identity);
            }
    }
}

```

- Inisialisasi Script MazeLogic seperti pada gambar dibawah. Pastikan Cube yang di pasang adalah GameObject yang berada pada **Assets/Prefabs**



- Running Program, jika pada saat program dijalankan menggenerate cube dengan luas 30x30, maka Script dan konfigurasi sudah benar. Jika belum lakukan pengecekan ulang !



3. Inisialisasi Tembok dan Lorong Maze.

Pada Langkah selanjutnya adalah melakukan inisialisasi Tembok dan Lorong Maze. Karena pada Labirin tidak mungkin semua areanya adalah tembok, dan tidak mungkin juga semua areanya adalah Lorong, akan selalu ada tembok dan Lorong yang akan digunakan player untuk eksplorasi. Berikut adalah Langkah langkahnya.

- Tuliskan Variable Global baru

```
public int width = 30; //x length
public int depth = 30; //z length
public GameObject Cube; //Maze Wall
public byte[,] map;
```

- Tuliskan 3 Method berikut

```
1 reference
void InitialiseMap() // initialise all Maps with 1
{
    map = new byte[width, depth];
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            map[x, z] = 1;    //1 = wall  0 = corridor
        }
}

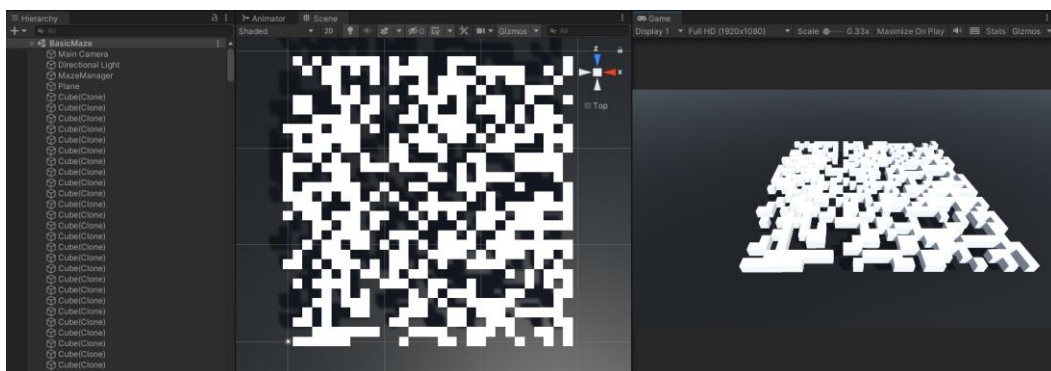
1 reference
void GenerateMaps() // initialise some corridors with randomness
{
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            if (Random.Range(0, 100) < 50)
                map[x, z] = 0;    //1 = wall  0 = corridor
        }
}

1 reference
void DrawMaps() // Draw All Value With GameObject
{
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            if (map[x, z] == 1)
            {
                Vector3 pos = new Vector3(x, 0, z);
                GameObject wall = Instantiate(Cube, pos, Quaternion.identity);
            }
        }
}
```

- Sesuaikan Method Start.

```
void Start()
{
    InitialiseMap();
    GenerateMaps();
    DrawMaps();
}
```

- Running Program, jika Maze terlihat memiliki lubang yang tergenerate secara random, maka script sudah benar, jika belum maka lakukan pengecekan ulang!



4. Drunken Crawler Algoritm.

Algoritma pertama yang akan dipelajari adalah Drunken Crawl Algoritm. Algoritma ini disebut Drunken (Mabuk) karena algoritma ini bergerak dengan sangat random atau acak. Berikut adalah Langkah langkahnya.

- Buat Script baru Bernama **CrawlerAlgoritm** didalam GameObject **MazeManager**.
- Pada Script **CrawlerAlgoritm** akan mengimplementasi **Inheritance** dari script **MazeLogic**

```
Unity Script | 0 references
public class CrawlerAlgoritm : MazeLogic
{
    ...
}
```

- Kembali ke Script **MazeLogic**, sesuaikan **Method Generate** seperti pada script berikut.

```

2 references
public virtual void GenerateMaps()
{
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            if (Random.Range(0, 100) < 50)
                map[x, z] = 0; //1 = wall 0 = corridor
        }
}

```

- Tulis Beberapa Baris Script dibawah, dan hapus Method **Start** dan **Update**.

```

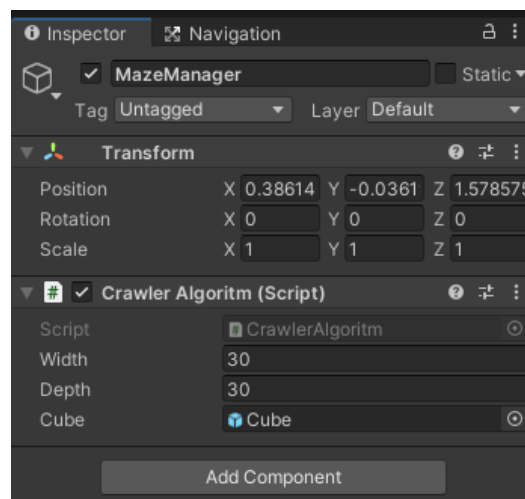
Unity Script (1 asset reference) | 0 references
public class CrawlerAlgoritm : MazeLogic
{
    2 references
    public override void GenerateMaps()
    {
        bool done = false;
        int x = width / 2;
        int z = depth / 2;

        while (!done)
        {
            map[x, z] = 0;
            x += Random.Range(-1, 2);
            z += Random.Range(-1, 2);

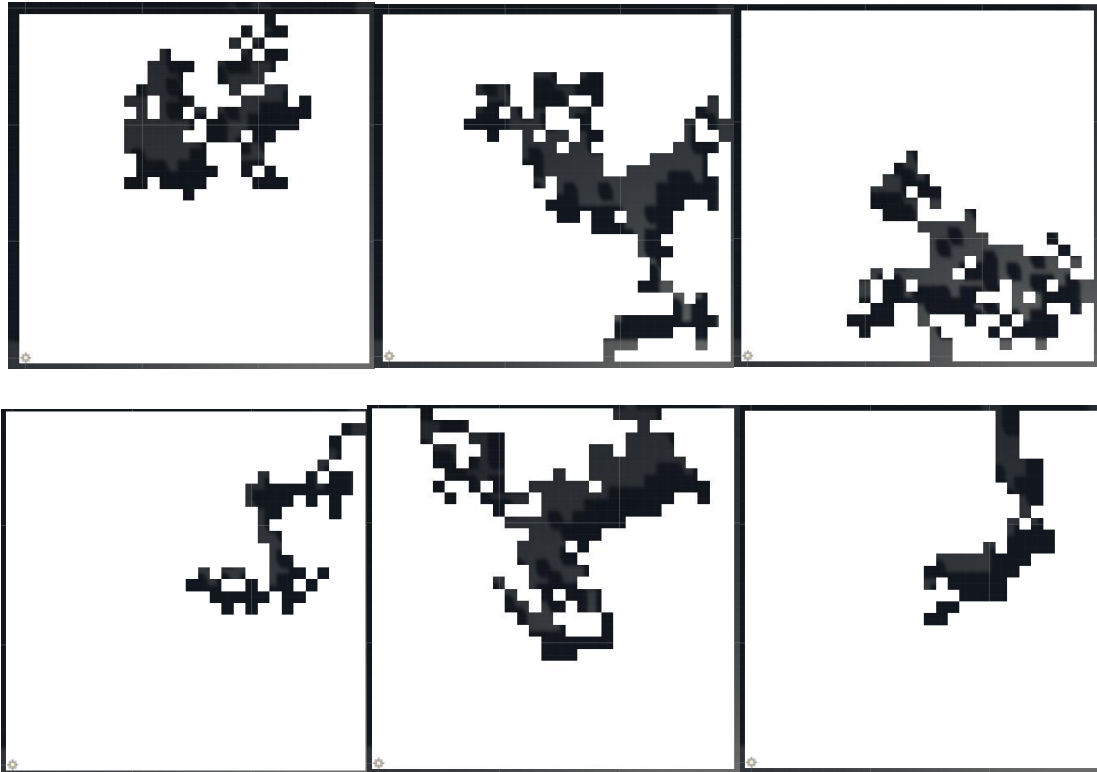
            if (x < 0 || x >= width || z < 0 || z >= depth)
                done = true;
            else
                done = false;
        }
    }
}

```

- Inisialisasi CrawlerAlgoritm seperti gambar dibawah dan remove Script MazeLogic dari Component GameObject MazeManager.



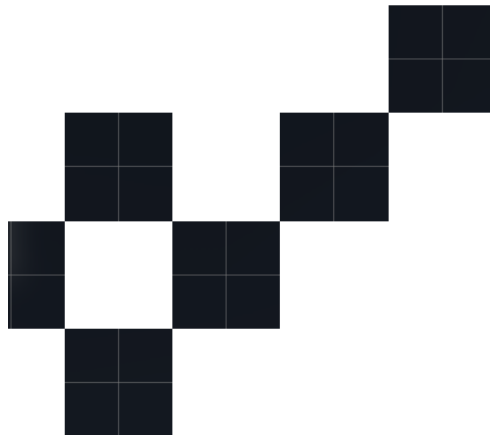
- Running Program, dan amati Maze yang tergenerate. akan terbentuk pola yang lebih terarah namun tetap tergenerate secara random. Pola yang akan terbentuk kurang lebihnya seperti pada gambar dibawah, jika pola masih belum sama, lakukan pengecekan ulang!.



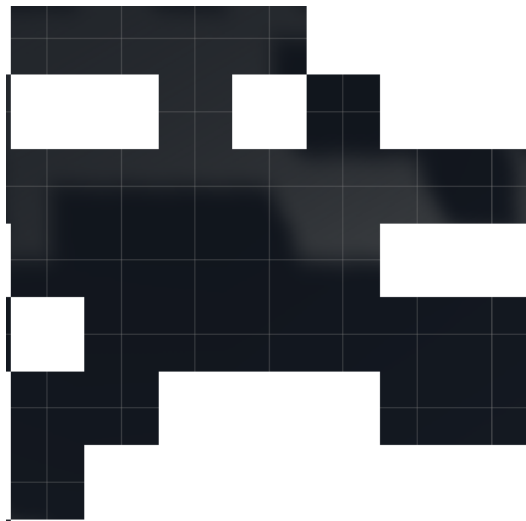
Pola Random yang terjadi pada Drunken Crawl ini adalah membuat titik start di Tengah Maze, lalu akan melakukan Langkah randomness ke arah manapun hingga salah menyentuh titik terluar maze.

5. Permasalahan Pada Drunken Crawl Saat ini.

Pada Langkah sebelumnya terlihat pola random maze sudah terbentuk, tetapi terdapat masalah pada Langkah ini, masalah tersebut adalah terbentuknya Lorong yang terhubung secara Diagonal. Hal tersebut merupakan masalah karena Lorong berbentuk diagonal tidak akan bisa dilewati oleh player. Berikut adalah contoh Lorong diagonal.



Player hanya dapat melewati Lorong yang terhubung secara horizontal dan vertical seperti pada gambar dibawah ini.



Pada bab selanjutnya, akan dijelaskan cara menghindari generate maze secara diagonal.

6. Drunken Crawl Avoid Diagonal Movement

Pada Langkah ini, akan dipelajari bagaimana membuat Maze hanya akan membuat Corridor Horizontal atau Vertikal saja, dan menghindari Lorong yang berbentuk diagonal. Berikut adalah Langkah langkahnya.

- Lakukan Penyesuaian Script seperti berikut.

```

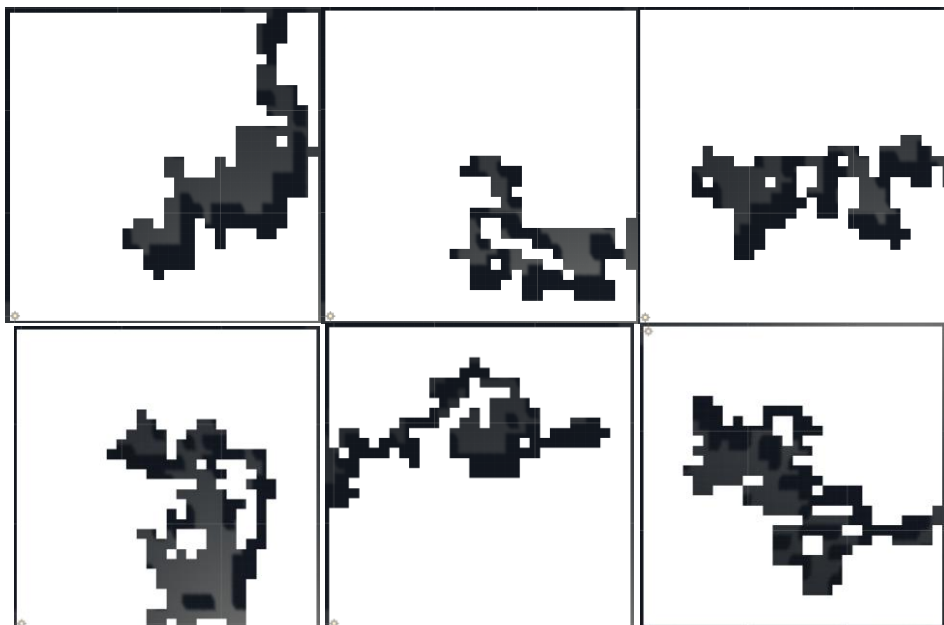
Unity Script (1 asset reference) | 0 references
public class CrawlerAlgoritm : MazeLogic
{
    2 references
    public override void GenerateMaps()
    {
        bool done = false;
        int x = width / 2;
        int z = depth / 2;

        while (!done)
        {
            map[x, z] = 0;
            if (Random.Range(0, 100) < 50)
                x += Random.Range(-1, 2);
            else
                z += Random.Range(-1, 2);

            if (x < 0 || x >= width || z < 0 || z >= depth)
                done = true;
            else
                done = false;
        }
    }
}

```

- Running Program, dan amati pola maze yang terbentuk, hasilnya tidak akan ada Lorong berbentuk diagonal seperti gambar dibawah ini. jika masih ada Lorong berbentuk diagonal, lakukan pengecekan ulang!



7. Selesai Untuk Praktikum Modul 8

Praktikum Dikatakan selesai jika sudah menyelesaikan Drunken Crawler Algoritm tanpa lorong Diagonal.

Tugas Modul 8

Maze Dungeon Environment Assets

1. Rules Modul 8

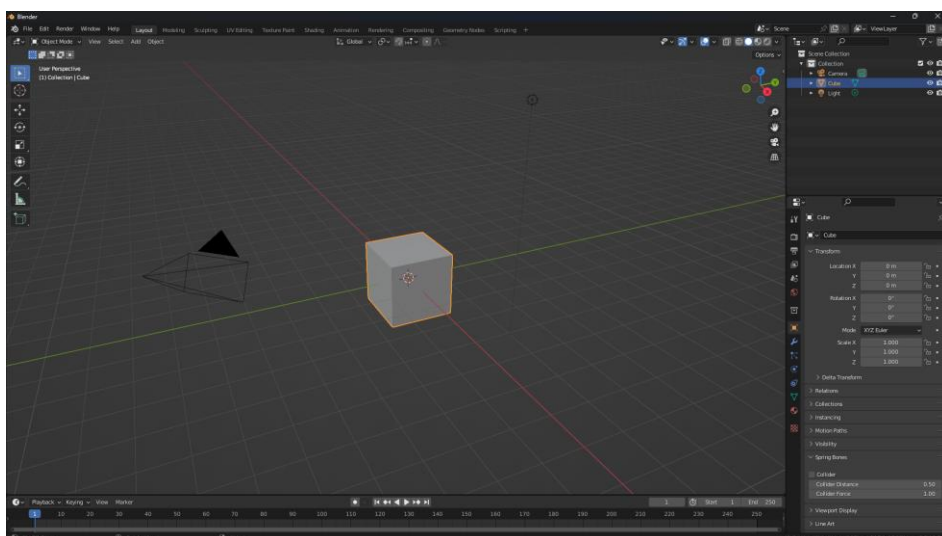
Tugas Modul 8 adalah melakukan proses Assets Swap dari Object Cube ke Object Building. terdapat Tutorial membuat building menggunakan blender 3D, namun bersifat Opsional (Step 2 - 5). Step yang wajib dikerjakan adalah Step 6 – 10. Asset yang dibuat pada step 2 – 5 dapat didownload di Drive Modul 8.

Building.rar

Jika ingin mengikuti cara pembuatan bangunan tersebut, dapat mengikuti step 2 – 5 dibawah ini.

2. Buka Blender 3D

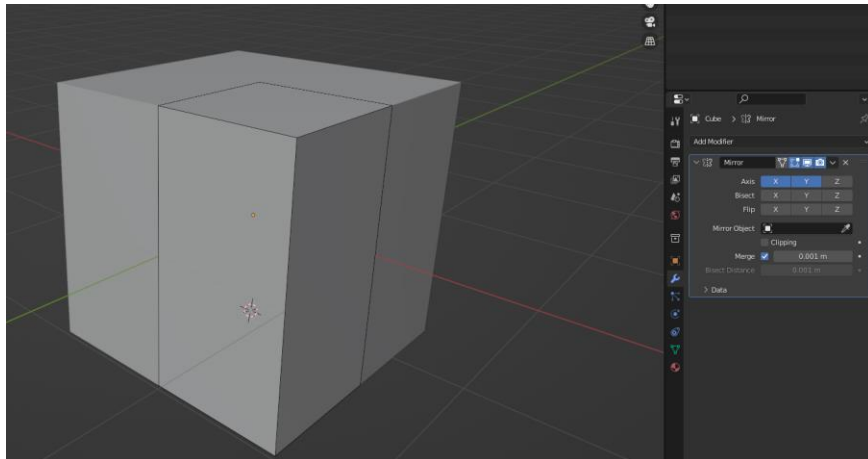
Langkah pertama pembuatan Environmant Assets adalah membuka software editor Blender 3D, jika belum menginstall program tersebut, download dan install blender 3D pada link berikut. (<https://www.blender.org/download/>). Berikut adalah tampilan awal blender3D.



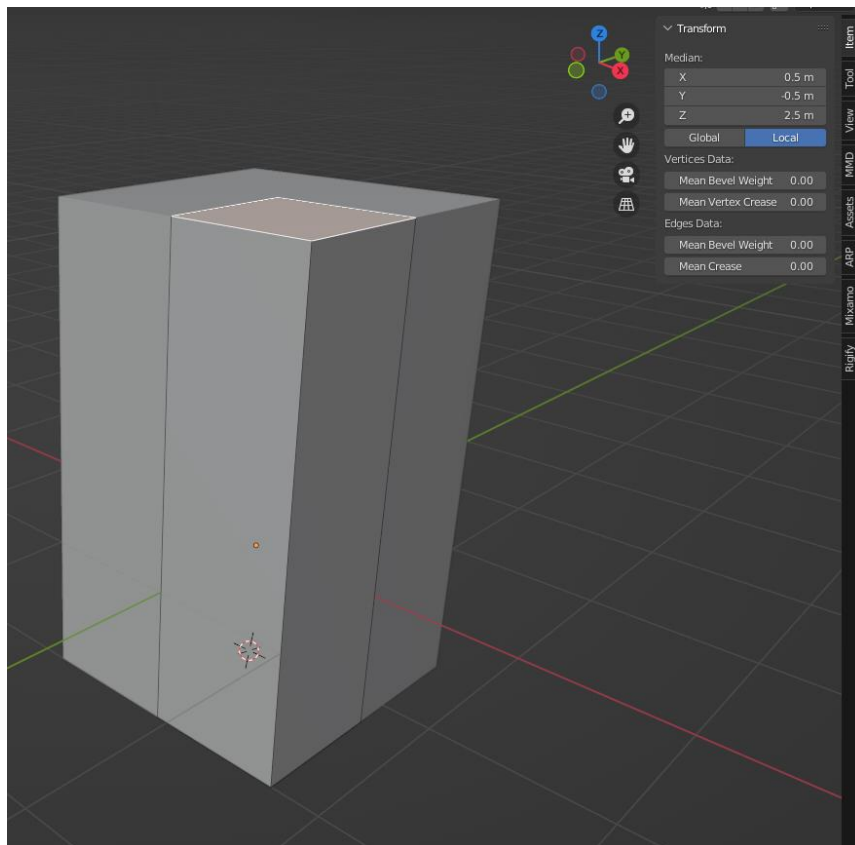
3. Proses Modelling.

Selanjutnya adalah proses modelling Assets, Langkah langkahnya adalah sebagai berikut.

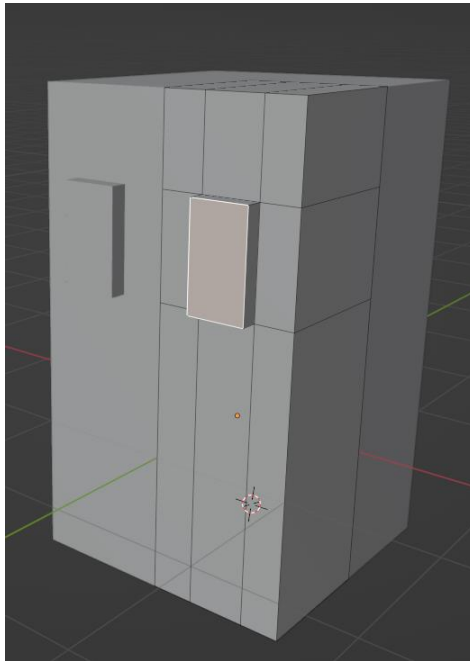
- Delete object camera dan light
- Mirror Object Dengan Sumbu X dan Y.



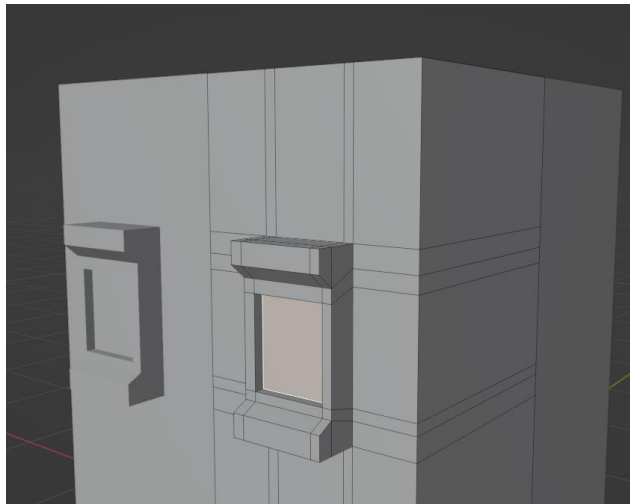
- Tarik atap atau sumbu Z hingga value 2.5, pastikan X dan Y masih tetap.



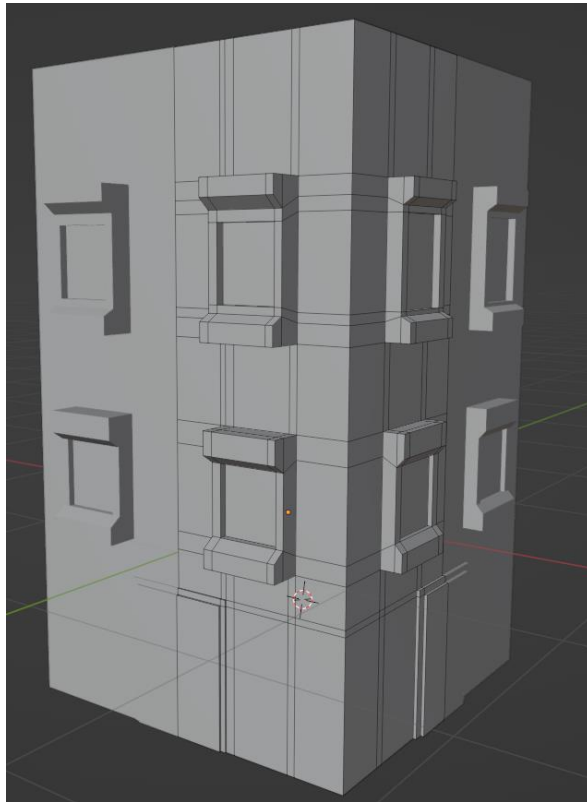
- Buat 2 Loopcut Horizontal dan 2 Loopcut Vertikal dan Extrude sisi Tengah.



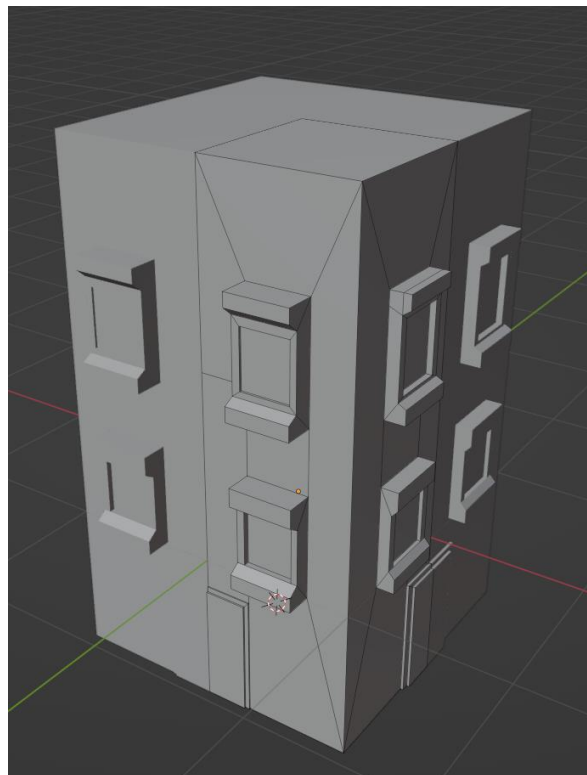
- Lanjutkan dengan pemodelan jendela.



- Lanjutkan dengan jendela yang lain dan pintu bagian Tengah bawah.

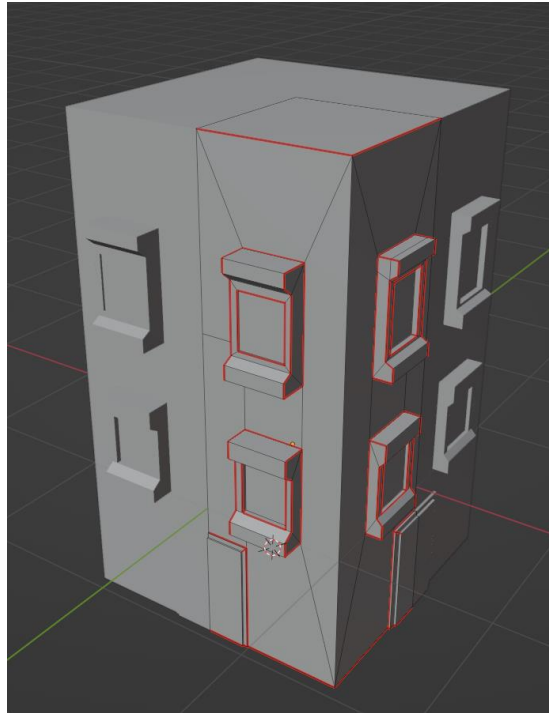


- (Opsional) Hapus beberapa vertex dan edge yang tidak dibutuhkan

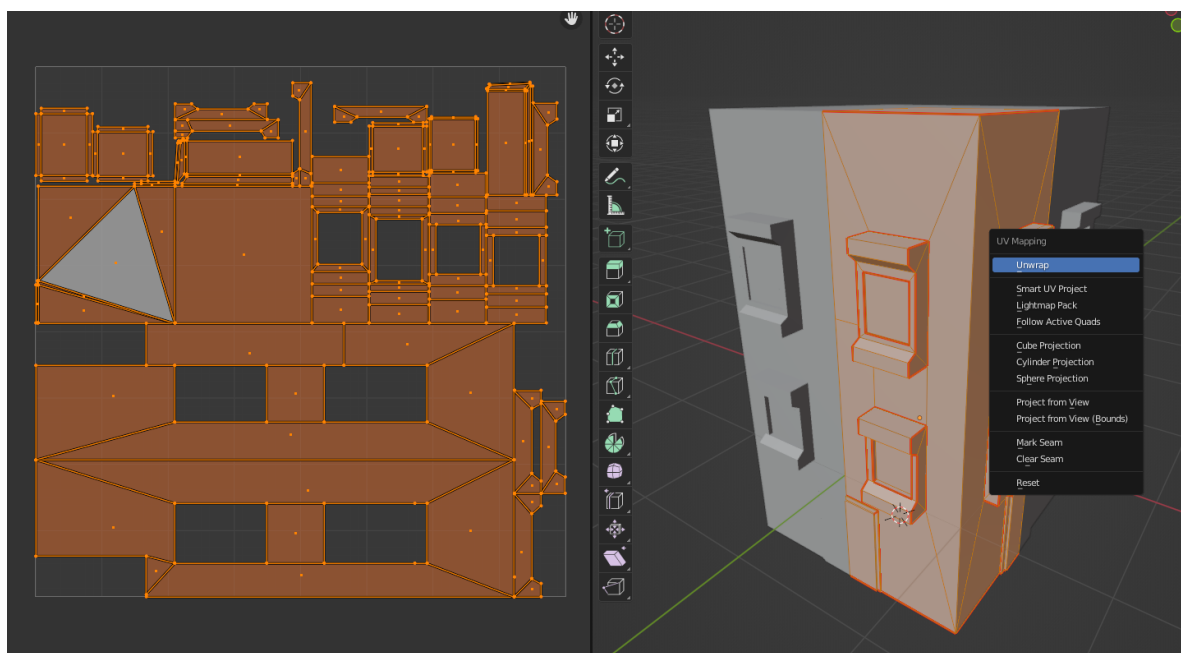


4. Proses UV Mapping

- Proses Markseam dengan menandai bagian UV yang ingin di potong. Proses tersebut bisa dilakukan dengan seleksi edge yang di inginkan, lalu tekan CTRL+E dan pilih Mark Seam.

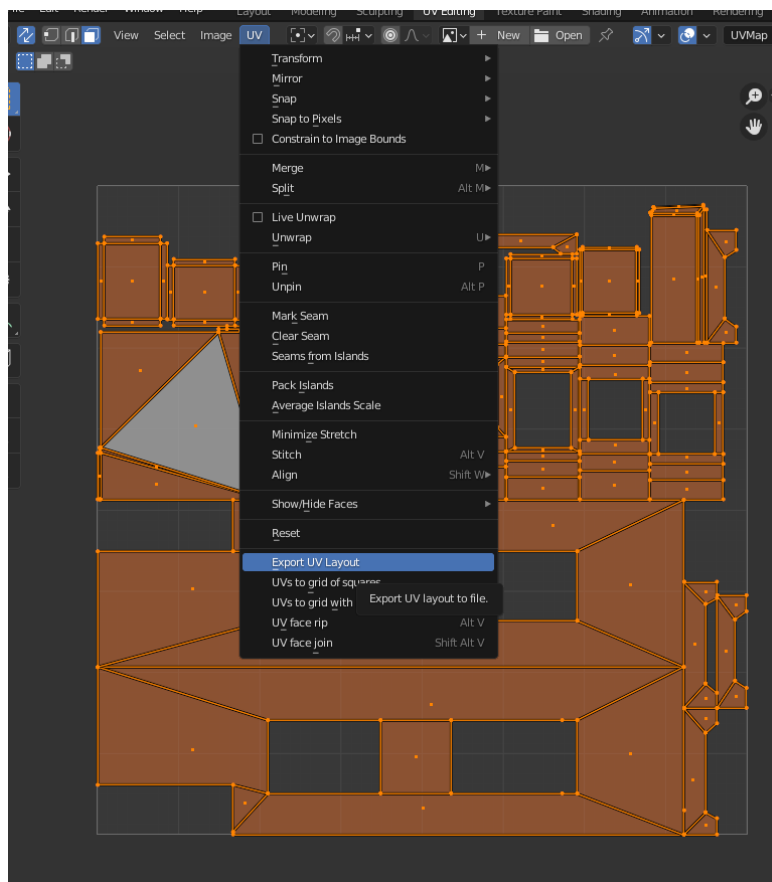


- Unwarp 3D model yang sudah di Mark Seam.



5. Texturing

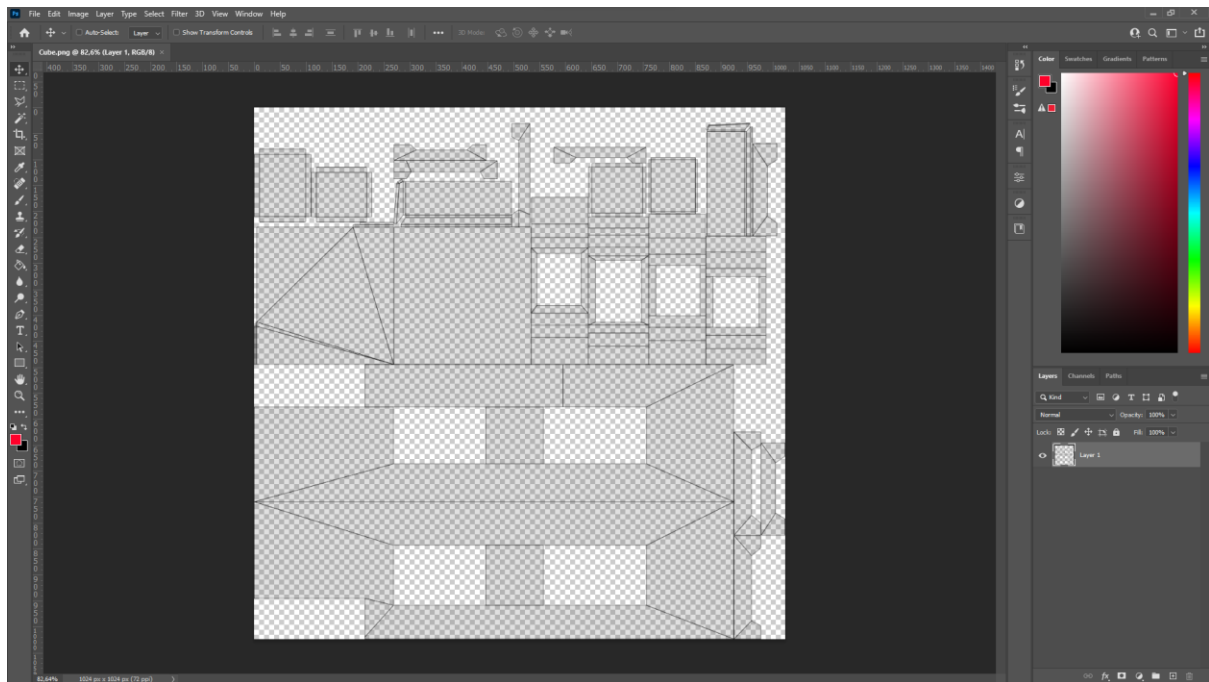
- Export UV Layout.



- Buka Software Editor 2D kesukaan kalian, bisa photoshop, Illustrator, PTS dll.



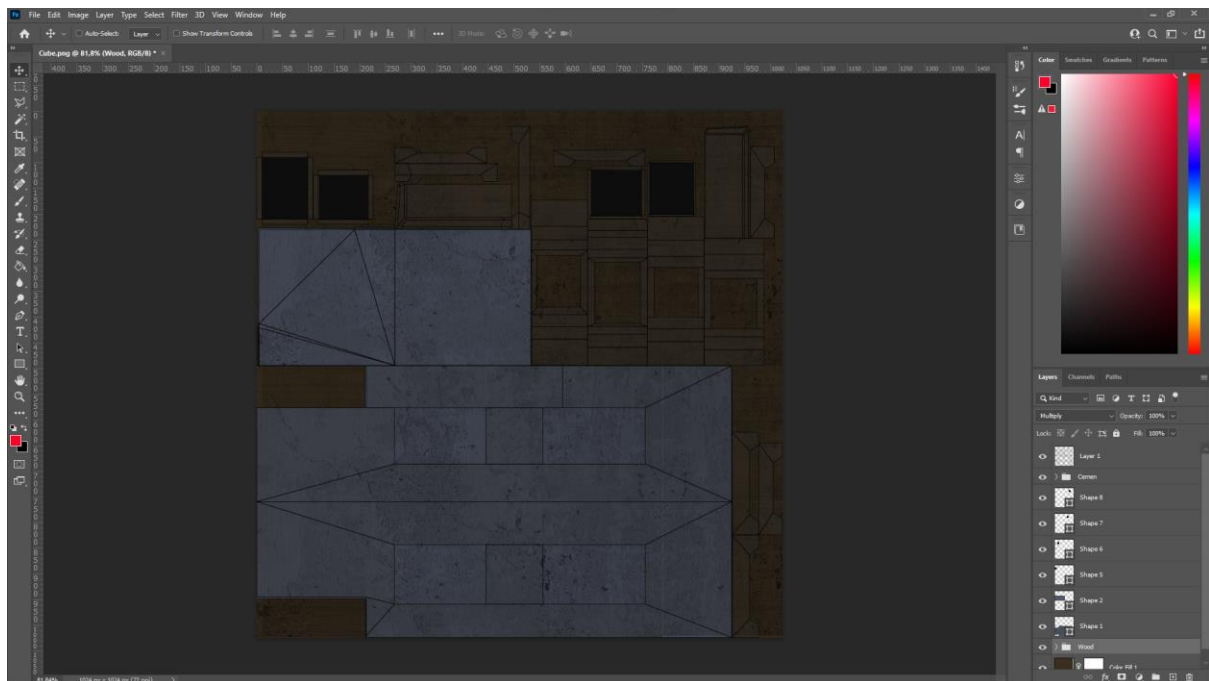
- Drag and drop UV Layout yang Sudah di export sebelumnya kedalam software editor 2D.



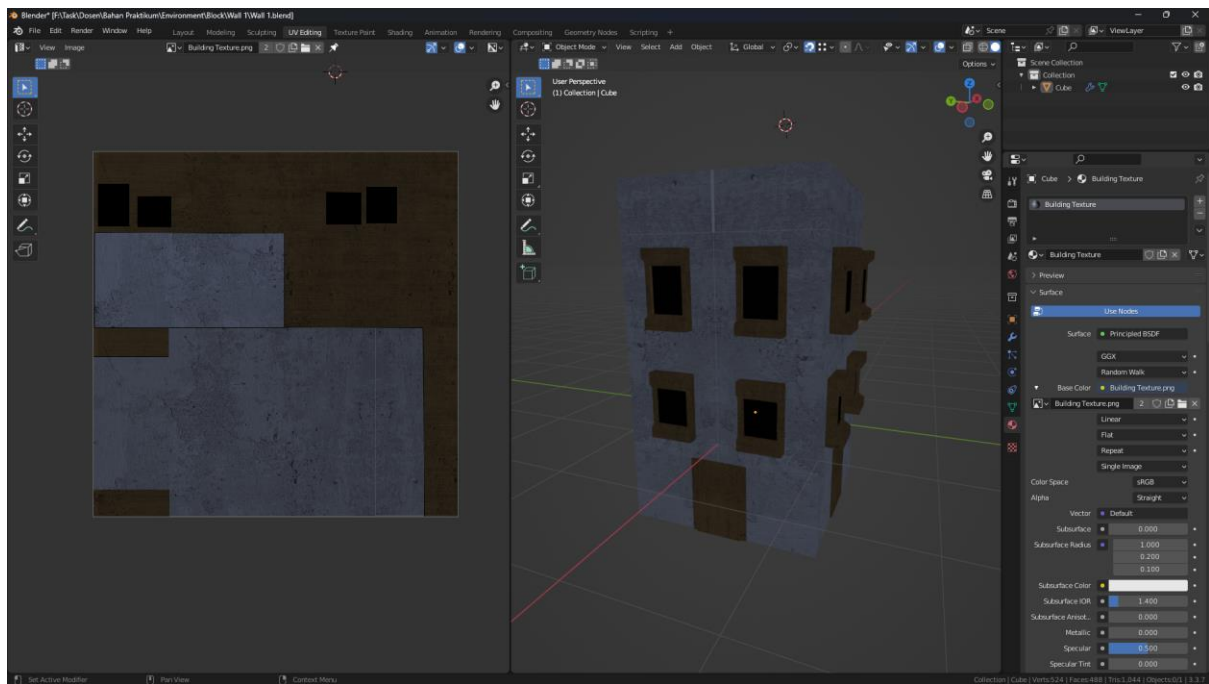
- (Optional) Download Pattern kayu dan Tembok di Drive Folder Modul 8.

 **Texture Pattern**

- Warnai Texture Sesuai dengan yang di inginkan.

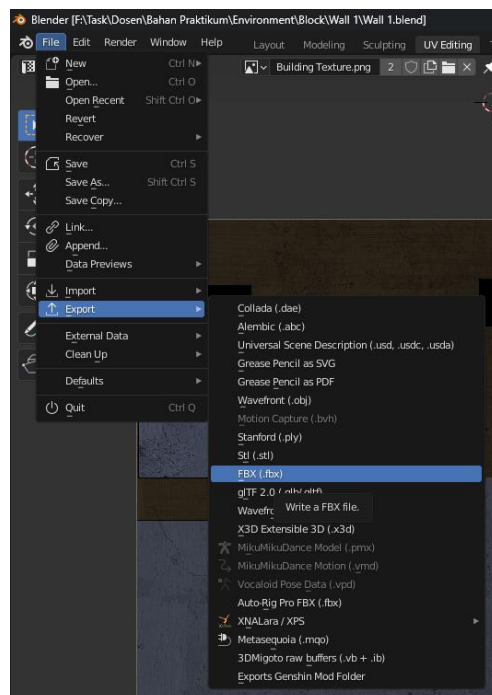


- Export Texture dan Import kedalam Blender

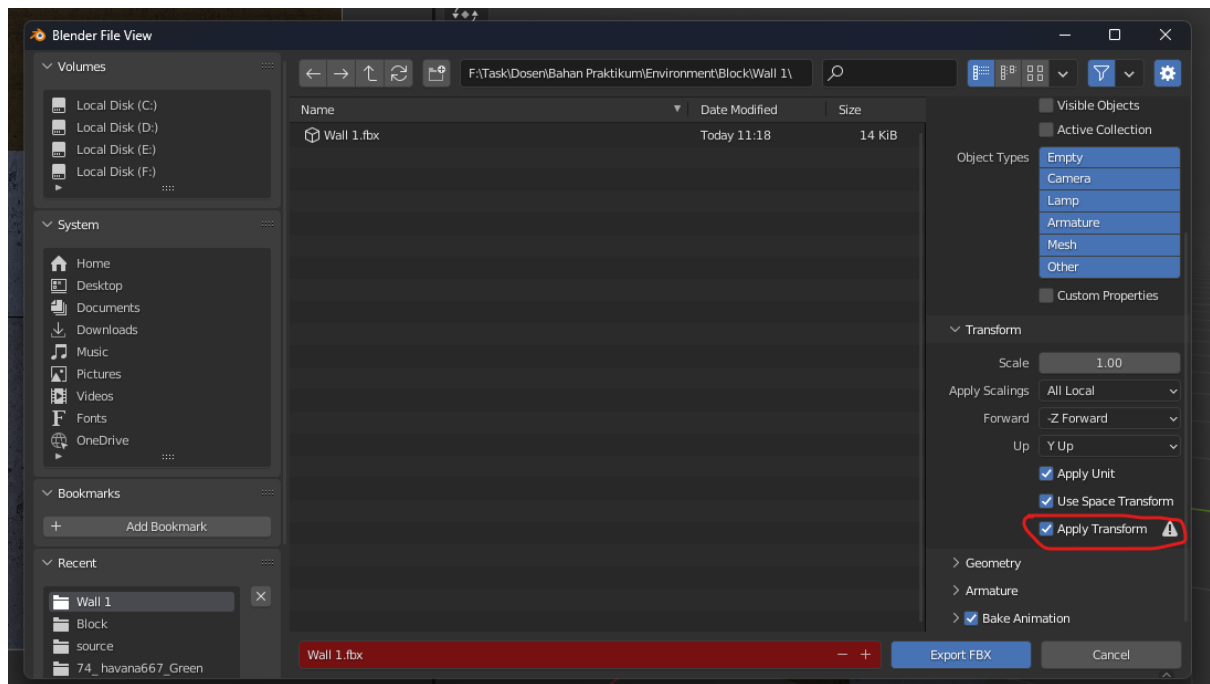


6. Blender to Unity

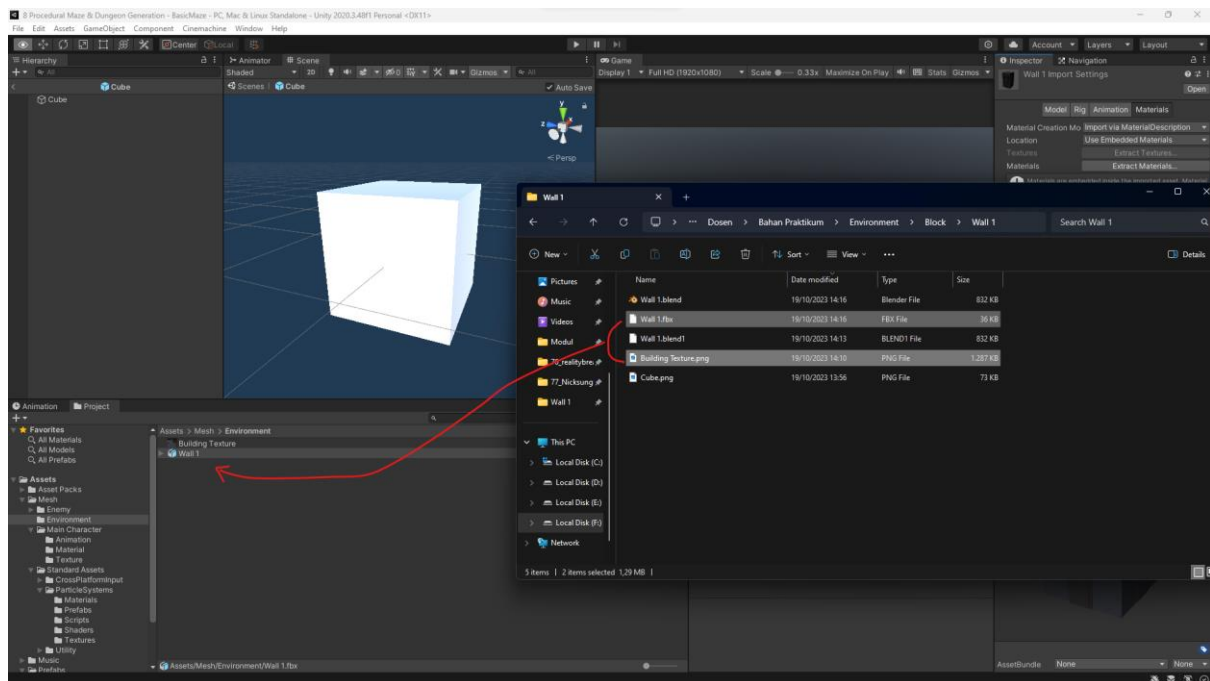
- Export File dengan format FBX



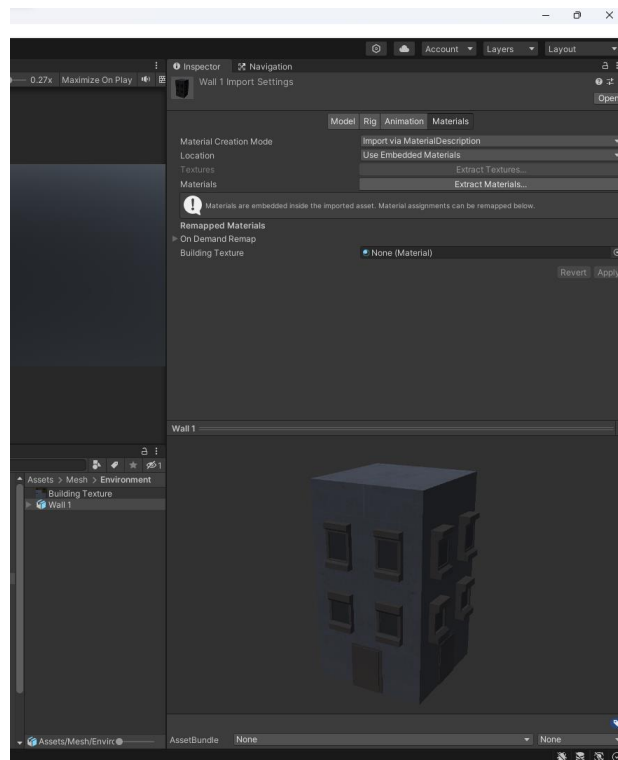
- Centang Apply Transform lalu Export File.



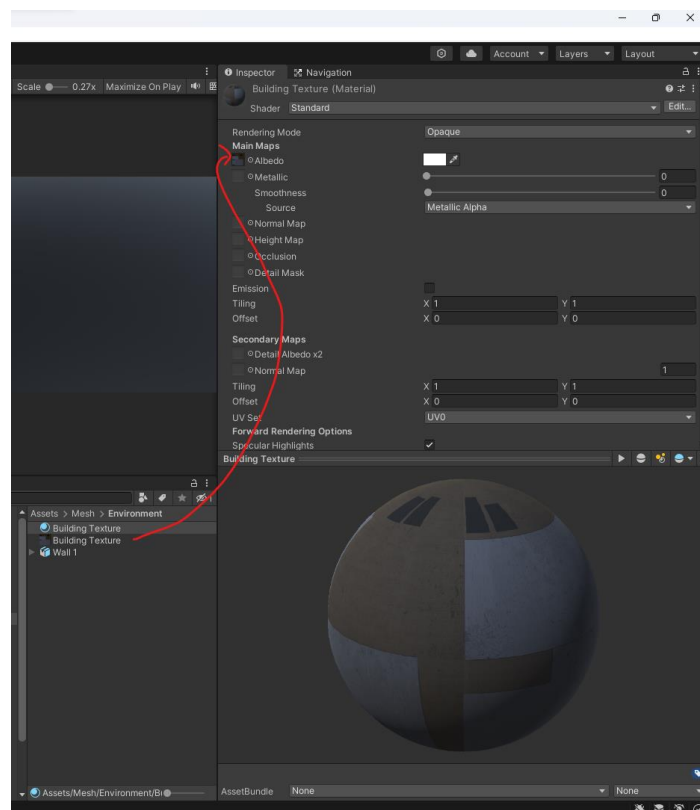
- Drag And Drop File FBX beserta Texturenya.



- Pastikan Texture sudah terpasang pada buiding.

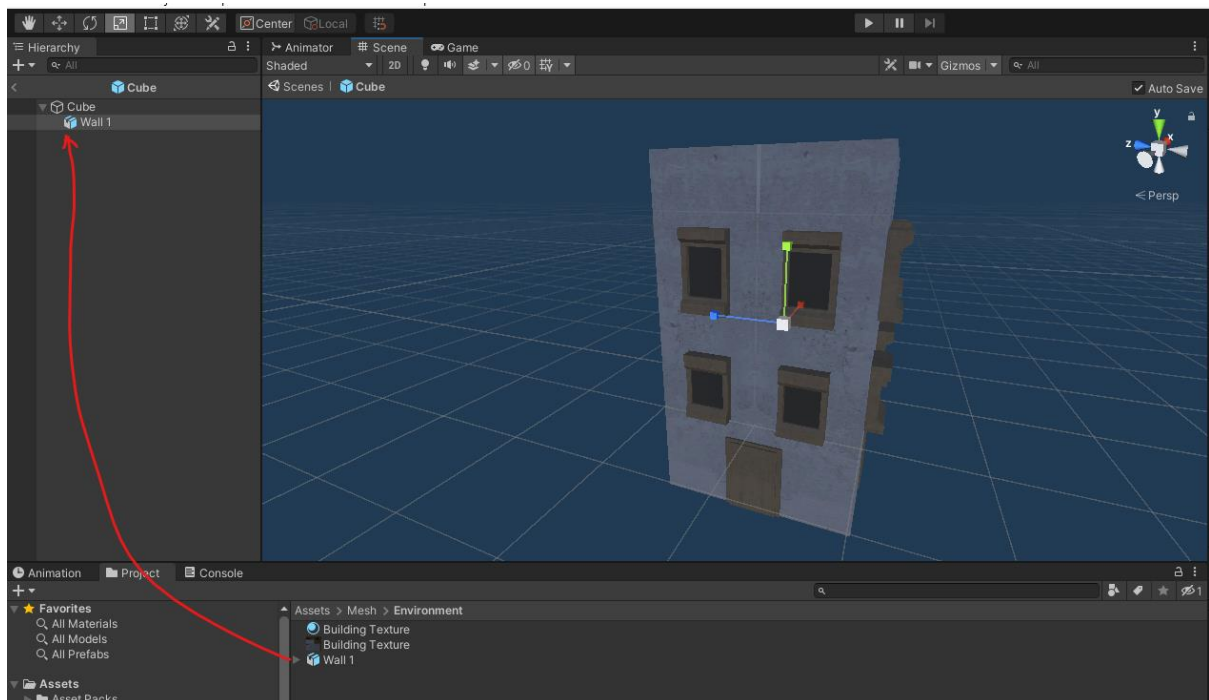
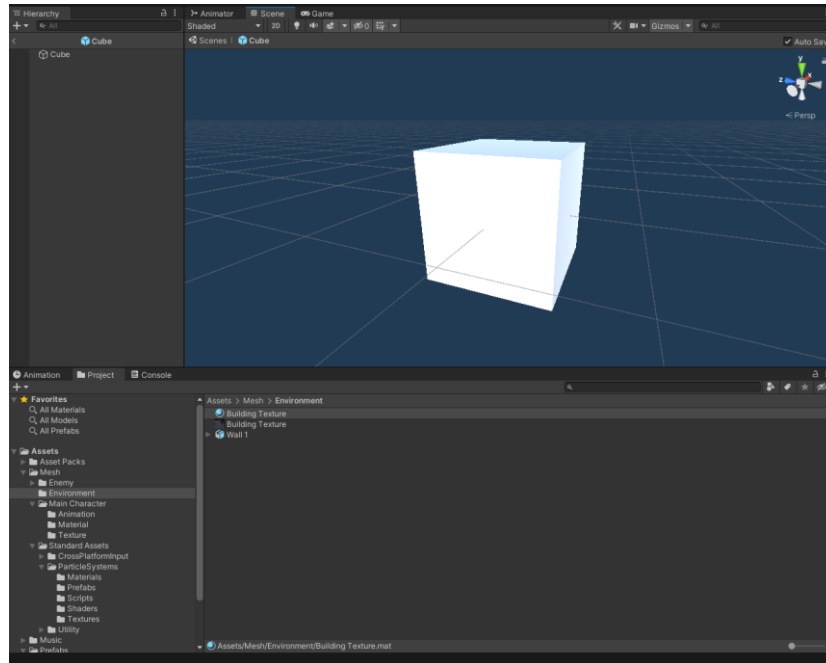


- Jika belum, tekan tombol extract material, lalu seleksi material yang sudah di extract dan drag texture building ke albedo.

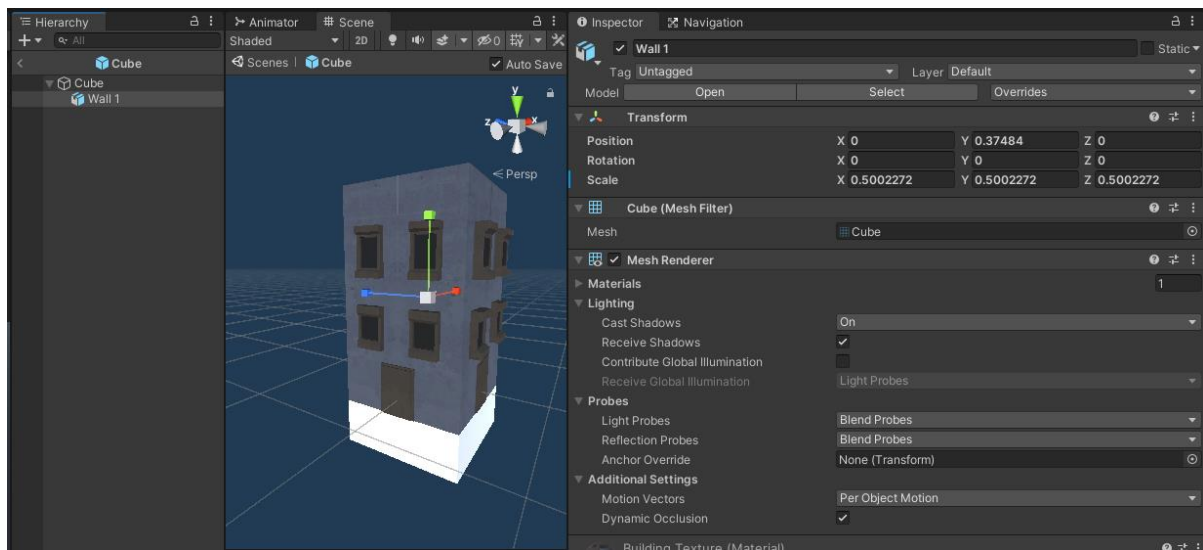


7. Replace Cube dengan Asset Building.

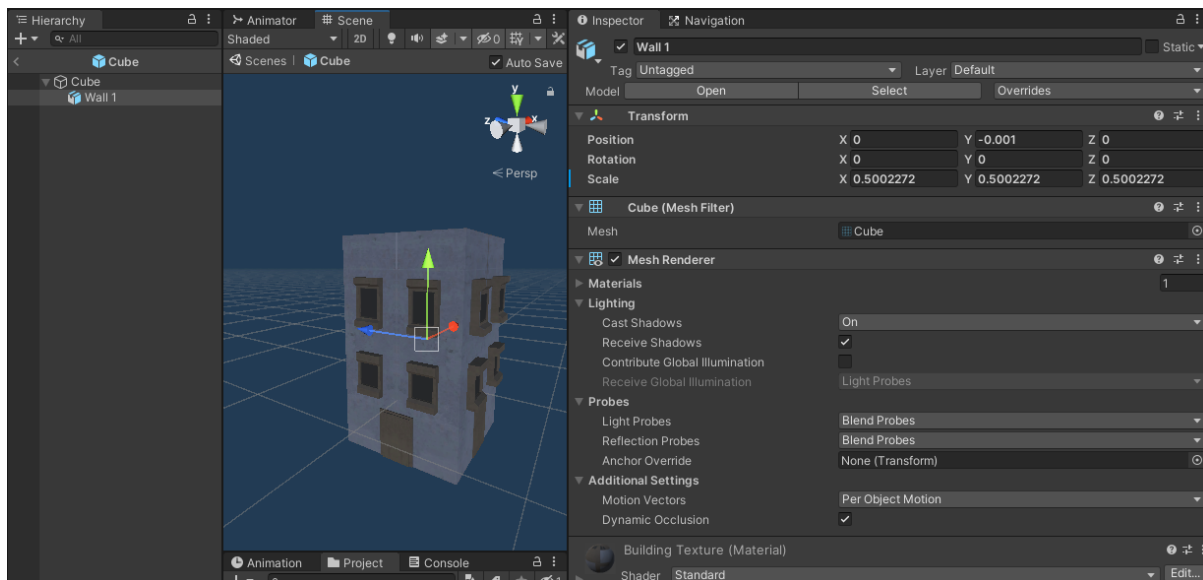
- Buka Prefabs Cube, dan Drag Building kedalam prefab Cube



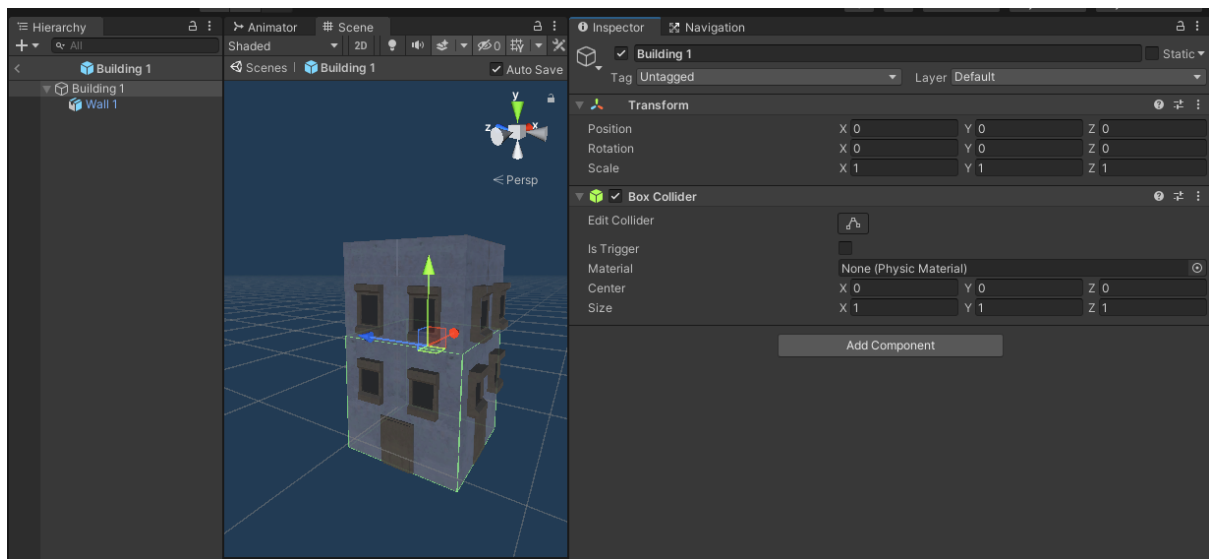
- Scale Down bangunan hingga Skala Z dan X sama dengan Cube.



- Adjust posisi bangunan agar titik terendah bangunan sama dengan titik terendah cube.

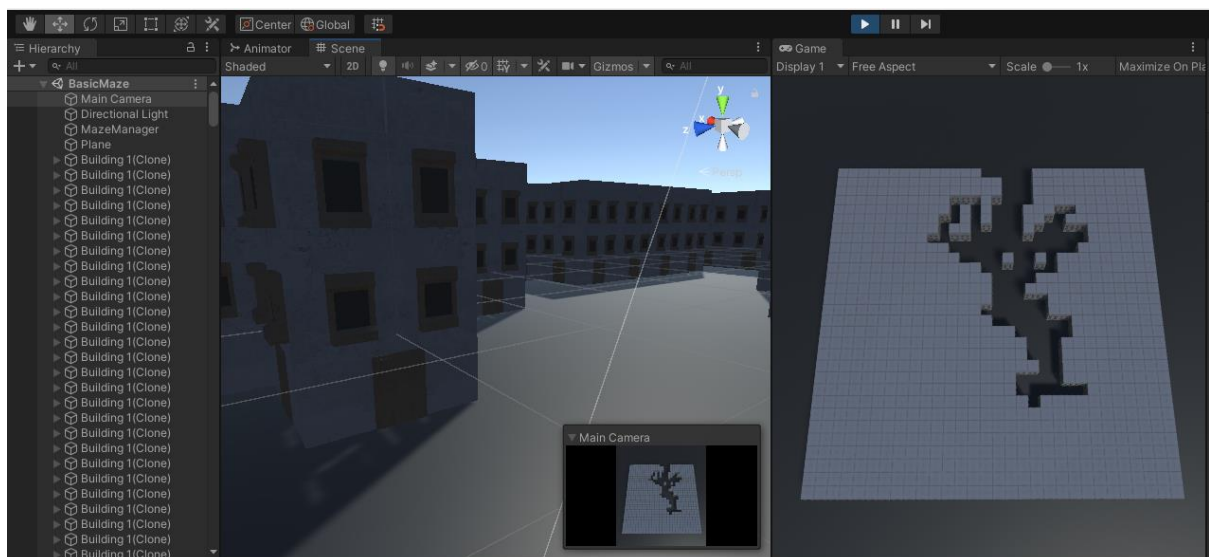


- Lanjut dengan menghapus component Mesh Filter dan mesh renderer pada GameObject Cube. Lalu rename menjadi Building 1.



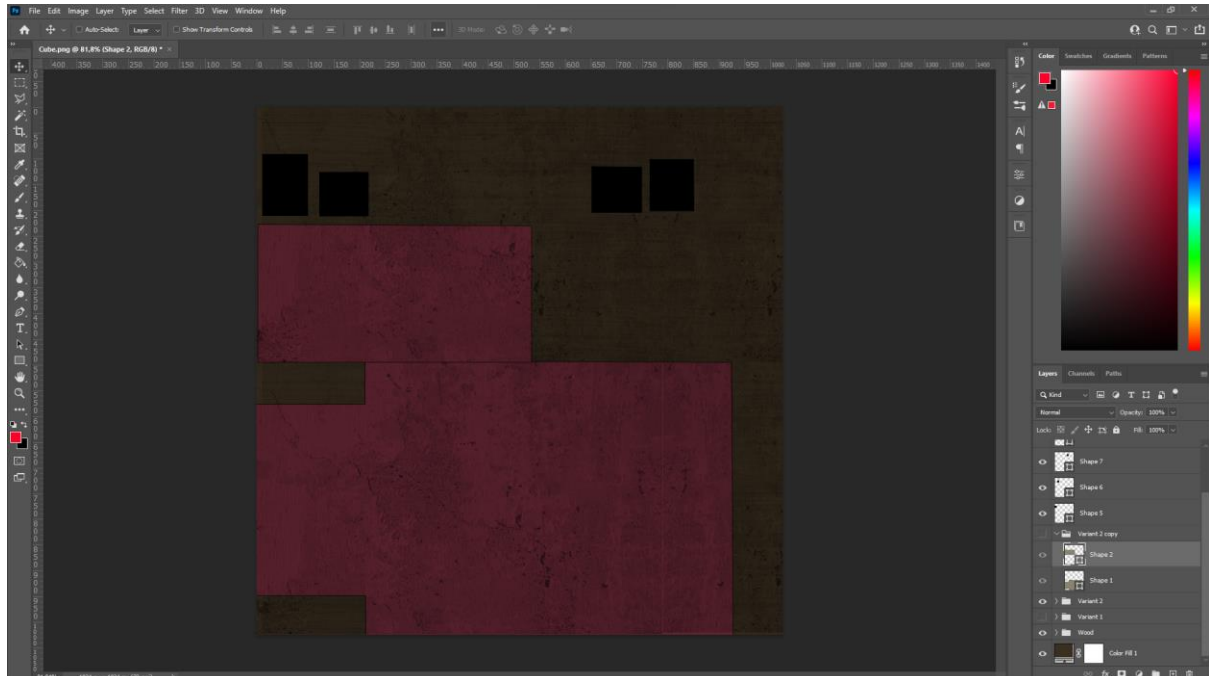
8. Testing Program dengan menggunakan Asset baru.

- Running Program. jika assets yang tergenerate secara otomatis sudah berubah, maka konfigurasi sudah benar, jika belum lakukan pengecekan ulang!.

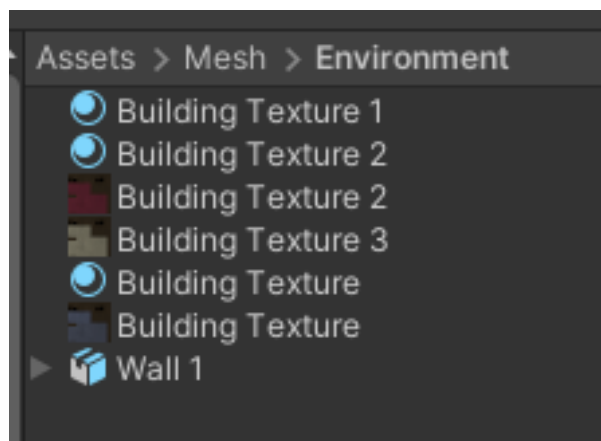


9. Dapat ditambahkan beberapa Variasi Warna agar tampilan tidak monoton

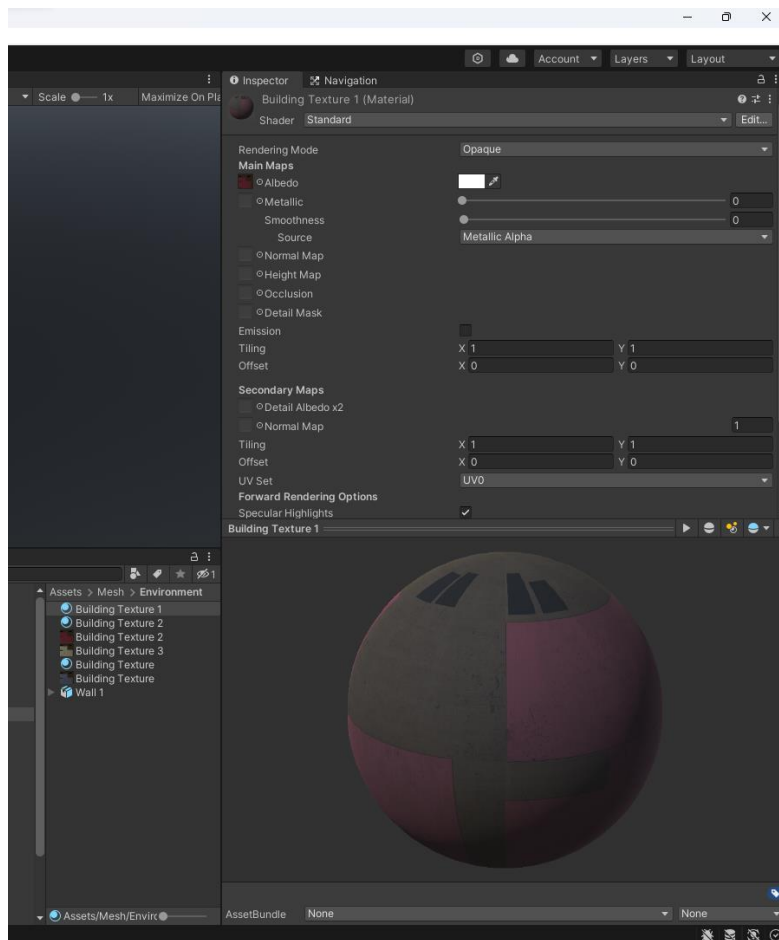
- Kembali ke Editor 2D dan Rubah Warna tembok bangunan menjadi beberapa bangunan.



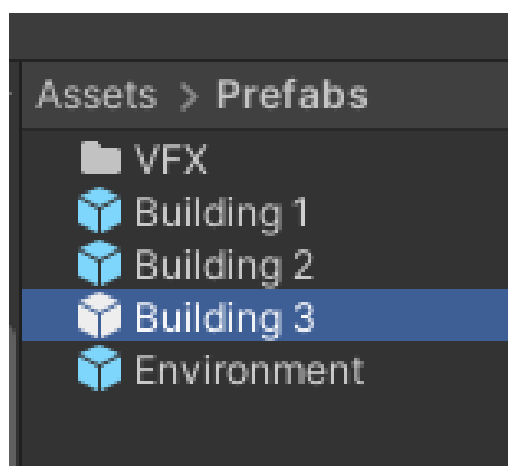
- Drag and drop semua texture baru ke unity dan duplicate Material sebanyak Variant warna yang dibuat.



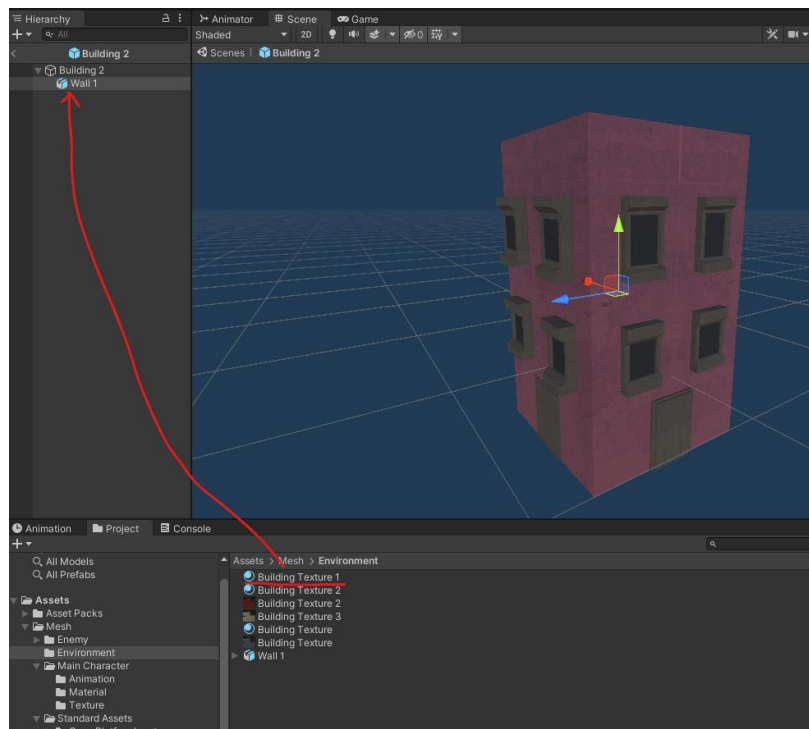
- Buka material building texture 1 dan drag texture building 2 kedalam albedo. Lakukan juga pada material building texture 2.



- Duplicate Prefabs Buiding 1 sebanyak variant yang dibuat



- Buka Building 2 dan drag and drop material variant 2 ke dalam GameObject Wall1.



- Lakukan Hal yang sama dengan Variant selanjutnya.
- Tulis Script berikut pada MazeLogic. Rubah format Cube menjadi List.

```

Unity Script | 1 reference
public class MazeLogic : MonoBehaviour
{
    public int width = 30; //x length
    public int depth = 30; //z length
    public List<GameObject> Cube; //Maze Wall
    public byte[,] map;
}

```

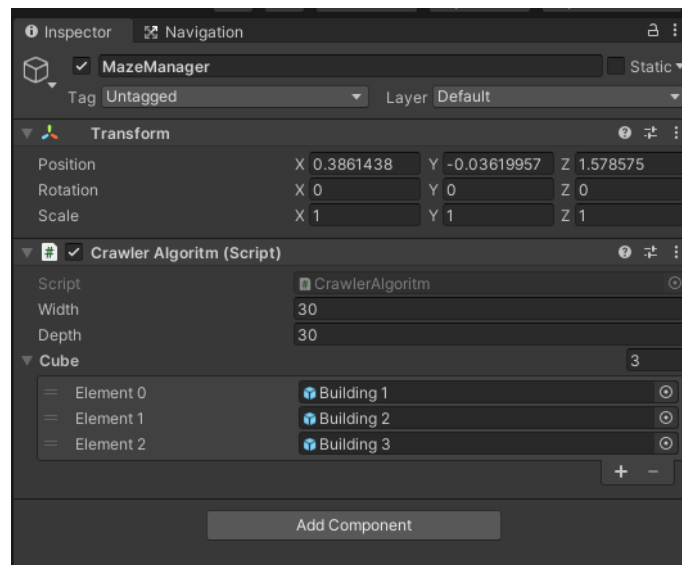
- Rubah 1 Baris pada Method DrawMaps.

```

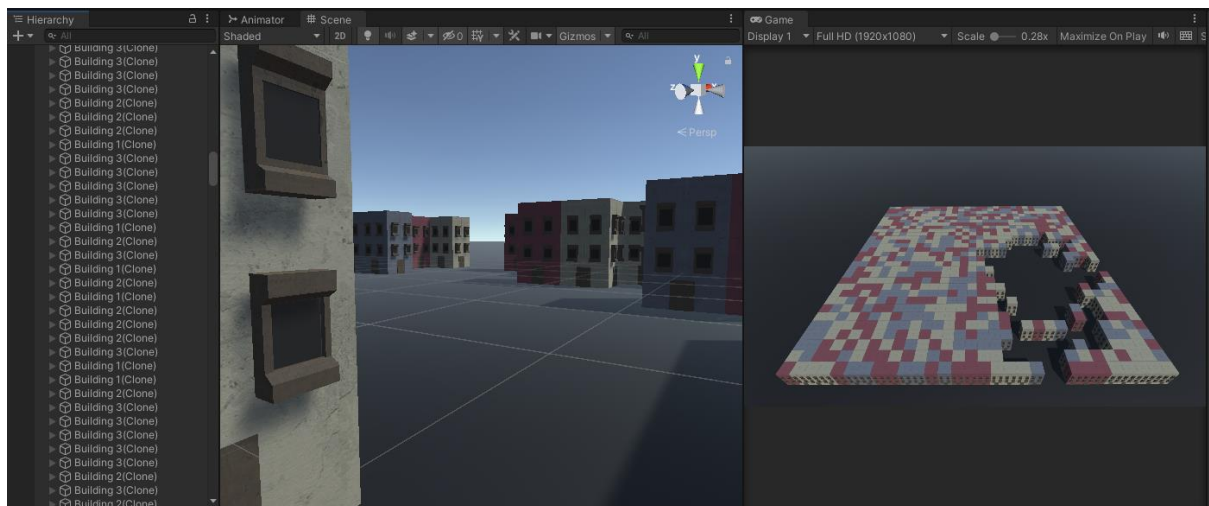
1 reference
void DrawMaps() // Draw All Value With GameObject
{
    for (int z = 0; z < depth; z++)
        for (int x = 0; x < width; x++)
        {
            if (map[x, z] == 1)
            {
                Vector3 pos = new Vector3(x, 0, z);
                GameObject wall = Instantiate(Cube[Random.Range(0, Cube.Count)], pos, Quaternion.identity);
            }
        }
}

```

- Inisialisasi Script pada GameObject MazeManager.



- Pengujian Running Program, jika Hasil Generate Maze sudah memunculkan semua variant bangunan, maka konfigurasi dan script sudah benar, jika belum lakukan pengecekan ulang!



10. Selesai untuk Tugas Modul 8

Tugas Modul 8 dikatakan selesai jika praktikan sudah mengganti object cube menjadi bangunan yang di design secara custom, dan juga sudah terdapat beberapa variant bangunan.